

Encoding Categorical (Technical) Data in Machine Learning

◆ 1. What is Technical (Categorical) Data?

- Technical data refers to non-numerical data that represents categories.
- Two main types:
 - Numerical data – e.g., height, age, temperature.
 - Categorical data – e.g., gender, nationality, education level.

◆ 2. Types of Categorical Data

1. **Nominal Categorical Data**
 - No inherent order or ranking between categories.
 - Examples: State (e.g., West Bengal, Maharashtra), engineering branches.
2. **Ordinal Categorical Data**
 - Has a meaningful order or ranking.
 - Examples: Education level (High School < Undergraduate < Postgraduate), Ratings (Poor < Good < Excellent).

◆ 3. Why Encode Categorical Data?

- Machine learning algorithms work with numerical data only.
- Categorical strings need to be converted to numbers before training.

◆ 4. Encoding Techniques

✓ A. Ordinal Encoding

- Suitable for ordinal data (data with a logical order).
- Example:
Column: Education = [High School, Undergraduate, Postgraduate]
Mapping:
 - High School → 0
 - Undergraduate → 1
 - Postgraduate → 2
- Use OrdinalEncoder from Scikit-learn.
- You must specify the correct order.

✓ B. One-Hot Encoding

- Suitable for nominal data (no natural order).
- Creates binary columns for each category.
- Example:
Gender → Male, Female
Becomes:
 - Male → [1, 0]
 - Female → [0, 1]
- Use OneHotEncoder from Scikit-learn.

✓ C. Label Encoding

- Primarily used for target variables (y/labels), not features.
- Converts categories into integers (like ordinal) but doesn't consider order.
- Use LabelEncoder from Scikit-learn.
- Example for classification labels:
Output classes → [Yes, No] → Yes = 1, No = 0

◆ 5. When to Use What?

Data Type	Use Encoding	Tool/Class
Ordinal (input)	Ordinal Encoding	OrdinalEncoder
Nominal (input)	One-Hot Encoding	OneHotEncoder
Categorical (target)	Label Encoding	LabelEncoder



Example - Ordinal Encoding

- Column: Education = [High School, Undergraduate, Postgraduate]
- Define order:
 - High School → 0
 - Undergraduate → 1
 - Postgraduate → 2
- Code (using OrdinalEncoder or manually with map):

```
df['Education'] = df['Education'].map({  
    'High School': 0,  
    'Undergraduate': 1,  
    'Postgraduate': 2  
})
```



Summary:

- Always convert categorical (technical) data to numeric before feeding into ML models.
- Choose encoding based on **data type** (ordinal/nominal) and **model requirements**.
- Use **Label Encoding** for target variables, not features.
- Understand data semantics before applying any encoding.



Project Overview

You are given a dataset with the following features:

```
[2]: df = pd.read_csv('customer.csv')  
df.head()
```

```
[2]:
```

	age	gender	review	education	purchased
0	30	Female	Average	School	No
1	68	Female	Poor	UG	No
2	70	Female	Good	PG	No
3	72	Female	Good	PG	No
4	16	Female	Average	UG	No

- Customer ID
- Gender
- Review (product review)
- Education (level of education)
- Purchased (whether a recommended product was purchased again: Yes/No)

The task is to **prepare this data for machine learning**, specifically a **classification problem** where:

- Purchased is the **target variable**.
- Features like Gender, Review, and Education are **categorical** and need encoding.



Understanding Categorical Variables

Types of Categorical Variables:

1. **Nominal (No natural order)**
 - Gender → Male, Female
 - Purchased → Yes, No
2. **Ordinal (Has a meaningful order)**
 - Review → Poor < Average < Good
 - Education → School < UG < PG



Data Encoding Strategy



Step 1: Select Relevant Columns

Ignore Customer ID and Gender for simplicity.

Keep:

- Review (Ordinal)
- Education (Ordinal)
- Purchased (Target, Nominal)

```
[4]: df = df.iloc[:,2:]
     df.shape
```

```
[4]: (50, 3)
```



Step 2: Train-Test Split

Always split your dataset before applying transformations to avoid data leakage.

```
[5]: from sklearn.model_selection import train_test_split
     x_train,x_test,y_train,y_test = train_test_split(
         df.drop('purchased',axis = 1),
         df['purchased'],
         test_size = 0.2,
         random_state = 0)
     x_train.shape , x_test.shape
```

```
[5]: ((40, 2), (10, 2))
```

```
[6]: x_train.head()
```

```
[6]:
```

	review	education
33	Good	PG
35	Poor	School
26	Poor	PG
34	Average	School
18	Good	School



Step 3: Ordinal Encoding

Use OrdinalEncoder for Review and Education.

```
[7]: from sklearn.preprocessing import OrdinalEncoder
```

making an object of OrdinalEncoder

IN THAT OBJECT WE WILL STORE CATEGORIES(list of values you want to encode)

```
[8]: oe = OrdinalEncoder(categories = [['Poor','Average','Good'], ['School','UG','PG' ]])
     oe.fit(x_train)
```

```
[8]:
```

```
OrdinalEncoder
OrdinalEncoder(categories=[['Poor', 'Average', 'Good'], ['School', 'UG', 'PG']])
```

```
[9]: x_train_encoded = oe.transform(x_train)
     x_test_encoded = oe.transform(x_test)
```

```
[17]: x_train_df = pd.DataFrame(x_train_encoded,columns = x_train.columns)
     x_train_df.head()
```

```
[17]:
```

	review	education
0	2.0	2.0
1	0.0	0.0
2	0.0	2.0
3	1.0	0.0
4	2.0	0.0

```
[11]: oe.categories
```

```
[11]: [['Poor', 'Average', 'Good'], ['School', 'UG', 'PG']]
```

✓ This ensures the correct order is maintained (e.g., Poor = 0, Good = 2).

⚙️ Step 4: Label Encoding for Target

Use `LabelEncoder` for Purchased (Yes/No → 1/0)

```
[12]: from sklearn.preprocessing import LabelEncoder
```

```
[13]: le = LabelEncoder()
```

```
[14]: le.fit(y_train)
le.classes_
```

```
[14]: array(['No', 'Yes'], dtype=object)
```

```
[15]: y_train_encoded = le.transform(y_train)
y_test_encoded = le.transform(y_test)
```

```
[16]: y_train = pd.DataFrame(y_train_encoded)
y_train.head()
```

```
[16]:    0
0   1
1   1
2   0
3   0
4   0
```

⚠️ `LabelEncoder` should only be used for the target variable, not for input features unless they are ordinal.

⚠️ Common Pitfalls to Avoid:

- Do not use `LabelEncoder` on nominal input features (like Gender).
- Always split data first, then apply encoders **only on training set**, and transform the test set.
- Make sure the **order of categories** is passed explicitly when using `OrdinalEncoder`.

📌 Summary

Feature	Type	Encoder Used
Gender	Nominal	Ignored (or OneHot)
Review	Ordinal	<code>OrdinalEncoder</code>
Education	Ordinal	<code>OrdinalEncoder</code>
Purchased	Nominal (Target)	<code>LabelEncoder</code>